RabbitMQ DLQ Demonstration Guide

Notification Service API · Docker Compose · separate WorkerService · RabbitMQ Management UI

> **Goal.** Demonstrate that a poison notification command is retried three times, then preserved in `nsa.notifications.bulk.dlq` instead of being lost or retried forever.

Architecture being demonstrated

```
POST /api/v2/notifications/bulk → API saves job → RabbitMQ main queue → Worker retry × 3 → dead-letter exchange → DLQ
```

The API and WorkerService run separately. RabbitMQ keeps the command durable outside the API process. A final failure is rejected without requeue and routed to the Dead-Letter Queue (DLQ).

Part 1 — Prepare the local environment

1 Open PowerShell in the repository root

```
Set-Location 'C:\Users\Full Scale\projects\Projects'
Get-Location
```

Why: all paths in the commands below are relative to this folder.

2 Start Docker Desktop and verify it

```
docker version
docker compose version
```

Why: the Week 3 stack uses Linux SQL Server and RabbitMQ images. Docker Desktop must be running in Linux-container mode before Compose can start the database, broker, API, and worker.

3 Create and complete the local `.env` file

```
Copy-Item notification-api/.env.example notification-api/.env
notepad notification-api/.env
```

Replace every blank value. For a local-only demo, use your own strong values or this working example:

```
WEEK3_SQL_PASSWORD=LocalSql!2026Demo
WEEK3_RABBITMQ_USERNAME=nsa_demo
WEEK3_RABBITMQ_PASSWORD=LocalRabbit!2026Demo
```

Setting	Why it is needed
`WEEK3_SQL_PASSWORD`	Creates/authenticates SQL Server's `sa` login for API and worker access.
`WEEK3_RABBITMQ_USERNAME`	Creates the broker user used by the API publisher, worker consumer, and Management UI.
`WEEK3_RABBITMQ_PASSWORD`	Authenticates that broker user everywhere it is needed.

> **Security:** `notification-api/.env` is local-only and ignored by Git. Do not commit it or display it while screen-sharing after passwords are added.

4 Validate before starting containers

```
docker compose `
  --env-file notification-api/.env `
  -f notification-api/compose.week3.yml `
  config --quiet
```

Expected result: no output and exit code 0.

> **If Compose reports a required variable is missing:** the file was copied but its values are still blank. Reopen `notification-api/.env`, set all three values from Step 3, save it, and validate again. Compose stops deliberately because it cannot create the broker user or authenticate the application without them.

5 Start and verify the stack

```
docker compose `
  --env-file notification-api/.env `
  -f notification-api/compose.week3.yml `
  up --build --detach

docker compose `
  --env-file notification-api/.env `
  -f notification-api/compose.week3.yml `
  ps
```

Expected result: SQL Server and RabbitMQ become healthy; API and worker start after dependencies are ready.

6 Sign in to RabbitMQ Management

1. Open `http://localhost:15672`.
2. Use `WEEK3_RABBITMQ_USERNAME` and `WEEK3_RABBITMQ_PASSWORD` from your `.env` file.
3. Open **Queues and Streams**.

Why: this UI is the visible evidence that the broker owns the failed message. Confirm `nsa.notifications.bulk.v1` and `nsa.notifications.bulk.dlq` exist.

Part 2 — Demonstrate retry and DLQ routing

7 Enable the local poison-message hook and recreate the worker

Run these commands in the same PowerShell window. The variable is session-only; if you close the window, set it again before recreating the worker.

```
$env:WEEK3_FAILURE_INJECTION_SUBJECT = '[week3-poison]'

docker compose `
  --env-file notification-api/.env `
  -f notification-api/compose.week3.yml `
  up --detach --force-recreate worker
```

Why: the worker receives `RabbitMq:FailureInjectionSubject` and deliberately fails only a pending item whose subject exactly equals `[week3-poison]`.

8 Open the worker-log terminal

Open a **second** PowerShell window at the repository root. Run:

```
Set-Location 'C:\Users\Full Scale\projects\Projects'

docker compose `
  --env-file notification-api/.env `
  -f notification-api/compose.week3.yml `
  logs --follow --tail 20 worker
```

Every line beginning with `worker-1 |` is a worker log. Keep this window open while you submit the poison job. `Ctrl+C` stops log-following only; it does not stop the worker container.

9 Submit a new poison job

Open `http://localhost:8080/swagger`. Execute `POST /api/v2/notifications/bulk` with one valid item whose subject is exactly `[week3-poison]`:

```
{
  "notifications": [
    {
      "recipientEmail": "demo@example.test",
      "channel": 1,
      "subject": "[week3-poison]",
      "body": "Controlled local DLQ demonstration."
    }
  ]
}
```

Record the returned job ID and correlation ID. The API should return `202 Accepted`.

10 Show the third attempt and DLQ

Attempt	Expected worker action
1	Intentional failure; command is republished with retry count `1`.
2	Intentional failure; command is republished with retry count `2`.
3	Job becomes `DeadLettered`; worker rejects without requeue.

Refresh RabbitMQ Management → **Queues and Streams** → `nsa.notifications.bulk.dlq`. Show the ready message. In Swagger, call `GET /api/v2/notifications/bulk/{jobId}` and show status `DeadLettered`.

What to say: “The command exhausted its bounded retry budget. It was rejected without requeue and RabbitMQ routed it through the dead-letter exchange to a durable DLQ for investigation and controlled replay.”

Troubleshooting: no third attempt or dead-letter message

If you see `unknown flag: --force-` and then `recreate` is not recognized: the option `--force-recreate` was split across two lines. Docker did not recreate the worker, so it did not receive the poison-message setting.

Fix: use the exact multi-line command in Step 7. A PowerShell continuation character (```) must be the final character on each continued line; do not add spaces after it.

If the log says `Acknowledged command ...`: that job was processed normally. Confirm the hook was enabled and submit a *new* job with the exact subject `[week3-poison]`.

To show relevant recent worker evidence after the test:

```
docker compose `
  --env-file notification-api/.env `
  -f notification-api/compose.week3.yml `
  logs --tail 200 worker |
  Select-String -Pattern 'Dead-lettering|retry|poison'
```

Expected result: after three attempts, a line reports that the command is being dead-lettered. Then the DLQ contains the retained message.

11 Disable the local test hook

```
Remove-Item Env:WEEK3_FAILURE_INJECTION_SUBJECT -ErrorAction SilentlyContinue

docker compose `
  --env-file notification-api/.env `
  -f notification-api/compose.week3.yml `
  up --detach --force-recreate worker
```

Do this after the demonstration so normal local jobs are no longer deliberately failed.

Evidence checklist

- Compose status shows API, worker, SQL Server, and RabbitMQ running.
- Worker log shows bounded retry and dead-lettering.
- RabbitMQ Management shows a ready message in `nsa.notifications.bulk.dlq`.
- Job-status endpoint shows `DeadLettered`.

Scope: this demonstrates durable queueing, bounded retries, and DLQ routing. It is at-least-once delivery; Outbox, Inbox/deduplication, and automated replay are later reliability work.